Personally, I'd use hex escapes, but you need to be aware this exists. Octal escape syntax comes with a warning: always use the full three digits, `\001` instead of `\1`, because there is some overlap with another regex feature called backreferencing. We'll get to backreferences in due course, but be aware that single-digit octal escapes can have unintended effects.

Unicode Properties

Unicode property escapes take the form `\p{xx}` (to assert the property exists for a character) and `\P{xx}` (to assert a property does not exist). The Unicode Standard assigns a variety of properties to characters as metadata attached to the codepoints defined in the Unicode data files that were mentioned earlier in this chapter. For example, the following line is taken from https://www.unicode.org/Public/15.1.0/ucd/UnicodeData.txt (Unicode compliant software is compiled against files such as this one)…

```
0020;SPACE;Zs;0;WS;;;;;N;;;;;
```

This is the Unicode definition for code point `U+0020`, which is the `SPACE` character (mapped to the spacebar on keyboards). The third field – `;Zs;` – is a "General Category" code describing the character, in this case `Zs` stands for "Separator, space". It is these codes that are combined with the Unicode property escape to describe a class of characters in the pattern:

```
$text = 'Highly irregular was the time I found a human foot in a
toaster oven';
$pattern = '/\p{Zs}/';
var_dump(preg_match_all($pattern, $text));
```

```
Output:
int(13)
```

The sample data in `$text` contains thirteen spaces, the pattern `/\p{Zs}/` therefore matches thirteen times, as we can see from the output. In addition to individual character properties, sets of codepoints are flagged as belonging to scripts (or "alphabets" if you prefer, though the term is contentious for certain writing systems). Here's an example: